Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Elektro- und Informationstechnik

Elektro- und Informationstechnik (Bachelor)

Laborbericht Regelungstechnik

Versuch 02 - Füllstandsregelung

Ruth Riebel

Gruppennummer:	1
Gruppenmitglieder:	Ruth Riebel (86074), Rishabh Venugopal (96535)
Betreuer:	Prof. Dr.-Ing. Dirk Feßler und Bernd Walenta
Datum:	27. April 2026

Inhaltsverzeichnis

1 Einleitung **3**

2 Aufgaben zur Vorbereitung **3**

2.1 Mathematischer Zusammenhang zwischen Zufluss und Füllstand 3

2.2 Übertragungsfunktion 3

2.3 Blockschaltbild 3

2.4 Stationäre Genauigkeit ohne Abfluss 4

2.5 Stationäre Genauigkeit mit Abfluss 4

3 Aufgabe 1: Behälter ohne Wasserablauf **4**

3.1 Simulink-Modell 4

3.2 Variation der Regelverstärkung K_r 5

3.3 Verhalten mit Führungsgrößensprung 5

4 Aufgabe 2: Behälter mit Wasserablauf **8**

4.1 Linearisiertes Ausflussgesetz mit P-Regler 8

4.2 Linearisiertes Ausflussgesetz mit PI-Regler 12

4.3 Ausflussgesetz nach Torricelli 17

1 Einleitung

Dieser Bericht behandelt den zweiten Versuch des Labores Regelungstechnik. Es soll eine Füllstandsregelung in *Matlab/Simulink* simuliert werden. Zwei Szenarien werden unterschieden: ein Behälter mit Zufluss aber ohne Abfluss und ein Behälter mit Zufluss und Abfluss. Für das zweite Szenario werden unterschiedliche Modelle betrachtet, um den Zusammenhang zwischen Zufluss und Abfluss mathematisch zu beschreiben.

2 Aufgaben zur Vorbereitung

2.1 Mathematischer Zusammenhang zwischen Zufluss und Füllstand

Der Behälter ist zylindrischer Form und hat eine kreisrunde Querschnittsfläche A in $[m^2]$ mit dem Durchmesser d in $[m]$. Die Regelgröße ist der Füllstand $h(t)$ in $[m]$. Der Zufluss ist $q_{zu}(t)$ in $[\frac{m^3}{s}]$
Für den Behälter ohne Abfluss gilt folgender mathematischer Zusammenhang:

$$\dot{h}(t) = \frac{1}{A} \cdot q_{zu}(t) \quad \Longrightarrow \quad h(t) = \frac{1}{A} \int q_{zu}(t) dt \tag{1}$$

2.2 Übertragungsfunktion

Die Laplace-Transformation der Gleichung (1) ergibt die Übertragungsfunktion $G(s)$. Der Zusammenhang der Funktionen im Laplace-Bereich ist in Abbildung 1 dargestellt.

$$G(s) = \frac{H(s)}{Q_{zu}(s)} = \frac{\mathcal{L}\{h(t)\}}{\mathcal{L}\{q_{zu}(t)\}} = \frac{1}{As} \cdot \frac{Q_{zu}(s)}{Q_{zu}(s)} = \frac{1}{As} \tag{2}$$

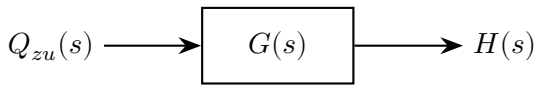

Abbildung 1: Signalflussdiagramm der Übertragungsfunktion $G(s)$

2.3 Blockschaltbild

Diese Übertragungsfunktion kann in einem Blockschaltbild dargestellt werden, wobei der Eingang $q_{zu}(t)$ durch einen Integrator mit der Verstärkung $\frac{1}{A}$ zum Ausgang $h(t)$ führt. Die Regelgröße ist die gewünschte Füllhöhe $h_{soll}(t)$, die mit der tatsächlichen Füllhöhe $h(t)$ verglichen wird, um den Fehler zu berechnen. Der Fehler wird von dem Regler verarbeitet, um die Stellgröße $q_{zu}(t)$ anzupassen und die gewünschte Füllhöhe zu erreichen. Das Blockschaltbild ist Grundlage für das Modell in *Simulink*, welches in Abbildung 2 gezeigt ist.

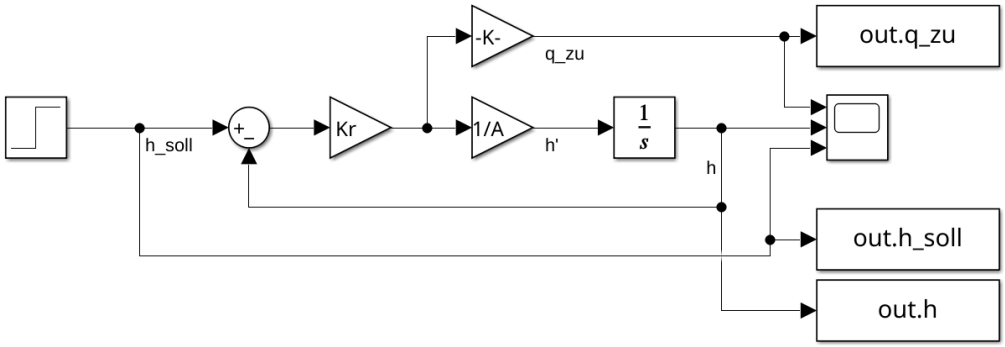

Abbildung 2: Blockschaltbild (Screenshot aus *Simulink*)

2.4 Stationäre Genauigkeit ohne Abfluss

Stationäre Genauigkeit bedeutet, dass die Regelgröße gegen den Sollwert konvergiert. Eine stationär genaue Regelung ist in der Lage, den Sollwert zu erreichen und zu halten. Es muss folgener mathematischer Zusammenhang gelten, damit die stationäre Genauigkeit erreicht wird:

$$\lim_{t\to\infty}(h_{soll}(t)) = \lim_{t\to\infty}(h(t)) \tag{3}$$

Mit Zufluss und ohne Abfluss wird die Füllhöhe kontinuierlich steigen, sie kann nicht stationär genau sein. Mathematisch betrachtet würde in diesem Fall der Regelgröße divergieren. In der Realität würde die Füllhöhe steigen, bis der Behälter überläuft.

2.5 Stationäre Genauigkeit mit Abfluss

In diesem Fall kann der Regler stationäre Genauigkeit erreichen. Wenn die einzuregelnde Füllhöhe überschritten wird, kann das Ventil des Abflusses geöffnet werden und die Füllhöhe sinkt wieder.

3 Aufgabe 1: Behälter ohne Wasserablauf

Dieser Teil simuliert eine Füllstandsreglung für einen Behälter ohne Abfluss. Der Behälter hat eine kreisförmige Querschnittfsläche mit einem Durchmesser von $D = 17\,\text{cm}$. Der Flächeninhalt berechnet sich zu $A = \pi r^2 = 22{,}7\,\text{mm}^2$. Der Zufluss $q_{zu}(t)$ ist die Stellgröße um den Füllstand $h(t)$ zu steuern. Das Ziel ist es, den Füllstand auf den gewünschten Sollwert $h_{soll}(t)$=10 cm zu bringen und dort zu halten.

3.1 Simulink-Modell

Das Simulink-Modell besteht aus einem Integrator, der den Füllstand berechnet, basierend auf dem Zufluss. Der Integrator hat eine Verstärkung von $\frac{1}{A}$, um den Zu-

sammenhang zwischen Zufluss und Füllstand zu berücksichtigen. Ein P-Regler wird verwendet, um die Stellgröße anzupassen, basierend auf dem Fehler zwischen dem Sollwert und dem tatsächlichen Füllstand. Der P-Regler und der Integrator sind die Regelstrecke. Das Blockschaltbild aus *Simulink* ist in Abbildung 2 abgebildet.

3.2 Variation der Regelverstärkung K_r

Die Regelabweichung e ist die Differenz zwischen Sollwert und Regelgröße.

$$e = h_{soll} - h \tag{4}$$

Um die Regelabweichung zu verarbeiten, wird diese mit dem einem Faktor K_r verstärkt. Dieser Faktor wird im Folgenden Reglerverstärkung genannt. Mit größeren Reglerverstärkungen reagiert der Regelkreis empfindlicher auf die Regelabweichung. Der Zufluss wird dementsprechend erhöht und der Sollwert ist schneller erreicht im Vergleich zu kleineren Reglerverstärkungen.

3.3 Verhalten mit Führungsgrößensprung

Für $t = 0$ wird ein Führungsgrößensprung auf h_{soll} =10 cm vorgegeben. Zu diesem Zeitpunkt ist der Behälter leer, dementsprechend ist die Regelabweichung maximal und der Regler öffnet das Ventil des Abflusses. Je mehr Wasser abgeflossen ist, desto höher ist der Füllstand und desto kleiner ist die Regelabweichung. Das Abflussventil wird immer weiter geschlossen, bis der Füllstand den Sollwert erreicht hat und das Ventil ganz schließt. Der Regelkreis wurde mit fünf verschiedenen Reglerverstärkungen modelliert. Die Zeitkonstante τ ist die Zeit, die der Regelkreis braucht, um 63.2% des Sollwerts zu erreichen. τ wurde für jede Reglerverstärkung bestimmt.

Der *Matlab*-Code für die Simulation und die Berechnung der Zeitkonstanten ist in 3.3.1 abgebildet. Der Code für die Erstellung der Plots und für die Ausgabe in der Konsole ist nicht dabei. Die Plots in Abbildung 3 zeigen die zeitlichen Verläufe der Führungsgröße, der Regelgröße und der Stellgröße für jede Reglerverstärkung. Die Tabelle 1 ordnet die Reglerverstärkungen ihren jeweiligen Zeitkonstanten zu.

Tabelle 1: Reglerverstärkungen und Zeitkonstanten

Reglerverstärkung K_R	Zeitkonstante τ [s]
0.010	0.10
0.008	0.11
0.006	0.17
0.004	0.17
0.002	0.24

Quellcode 3.3.1: *Matlab*-Code zur Simulation und Berechung der Zeitkonstanten

```
% Regelgröße: h , Führungsgröße: h_soll , Stellgröße: q_zu
Kr_vector = [0.002, 0.004, 0.006, 0.008, 0.01]; % reglerverstärkungen
n_sim = length(Kr_vector);
T_vector = zeros(1, n_sim);             % initialvektor für zeitkonstanten

for i = 1:n_sim
    Kr_actual = Kr_vector(i);
    assignin('base', 'Kr', Kr_actual);
    set_param('modell_ohne_abfluss/Step', 'After', num2str(0.10));
    simOut = sim("modell_ohne_abfluss.slx");

    if exist('simOut', 'var')
        if isstruct(simOut)
            h_soll = simOut.h_soll;
            h = simOut.h;
            q_zu = simOut.q_zu;
        else
            h_soll = simOut.get('h_soll');
            h = simOut.get('h');
            q_zu = simOut.get('q_zu');
        end
    end

    t_value = (0:length(h)-1)' * 0.01;
    h_value = h;
    h_soll_value = h_soll;
    q_zu_value = q_zu;

    sim_data{i}.t_value = t_value;
    sim_data{i}.h_value = h_value;
    sim_data{i}.h_soll_value = h_soll_value;
    sim_data{i}.q_zu_value = q_zu_value;
    sim_data{i}.Kr = Kr_actual;

    % zeitkonstante
    h_end = h(end);
    h_soll_end = h_soll(end);
    zielwert = 0.632 * h_end; % 63.2 % der Führungsgröße
    zielindex = find(h >= zielwert, 1, 'first');
    T_actual = t_value(zielindex);
    T_vector(i) = T_actual;
    fprintf('Reglerverstärkung Kr= %.3f:', Kr_actual);
    fprintf('    Zeitkonstante T= %.3f Sekunden\n\n', T_actual);

end
```

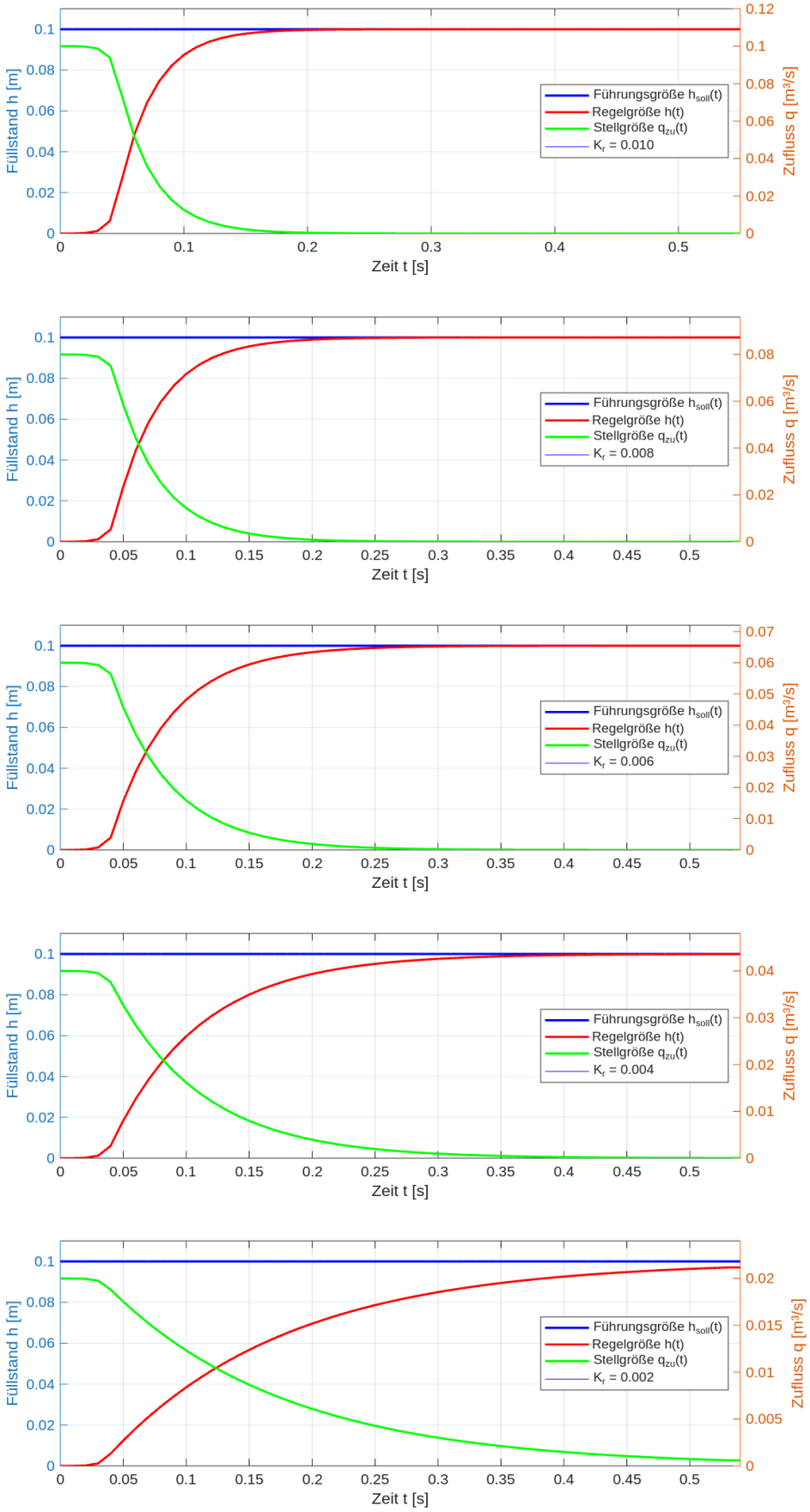

Abbildung 3: *Matlab*-Plots für verschiedene Reglerverstärkungen

4 Aufgabe 2: Behälter mit Wasserablauf

Um den Sollwert stationär genau regeln zu können, wird im Folgenden ein Behälter mit Abfluss betrachtet. Der Abfluss $q_{ab}(t)$ in $[\frac{m^3}{s}]$ ist für den Regelkreis eine Störung. Im Modell wird diese nach der Regelstrecke abgezweigt und vor die Regelstrecke rückgekoppelt. Die Störung beeinflusst die Regelgröße h. Der Regelkreis muss mit einer Änderung des Zuflusses reagieren, um die Abweichung auszugleichen.

4.1 Linearisiertes Ausflussgesetz mit P-Regler

Das erste Modell geht vereinfachend davon aus, dass der Abfluss proportional zur Füllhöhe ist. Der mathematische Zusammenhang ist in Gleichung (5) dargestellt.

$$q_{ab}(t) = c \cdot h(t), \quad c = 3\frac{l}{sm} \tag{5}$$

Für dieses Modell wird das Modell in Abbildung 2 um das Signal q_{ab} erweitert, welches vor die Regelstrecke rückgekoppelt wird. Der effektive Zufluss q_{eff} berücksichtig nun, dass das Wasser direkt wieder abfliesst. Vor der Regelstrecke wird q_{eff} wie in Gleichung (6) berechnet.

$$q_{eff} = q_{zu} - q_{ab} \tag{6}$$

Der Regelkreis ist nicht stationär genau. Die Regelgröße nähert sich der Führungsgröße an. Die Stellgröße wird solange kleiner und die Störgröße größer, bis gilt: $q_{zu} = q_{ab}$. In diesem Fall ist der effektive Zufluss Null. Da aber der effektive Zufluss das Eingangssignal der Regelstrecke ist, wird die gesamte Regelstrecke zu Null gesetzt. Auch die Rückkopplung und damit die Regeldifferenz liefern nun den Wert Null. Aus Sicht des Regles heißt eine Regeldifferenz von Null, dass die Regelgröße gleich der Führungsgröße ist. Damit ist die Regelung abgeschlossen, obwohl der gewünschte Füllstand noch nicht erreicht wurde. Der tatsächliche Füllstand heißt stationärer Endwert. Die Differenz zwischen der gewünschten Sollhöhe und dem stationären Endwert nennt man die absolute Regelabweichung. Je größer die Reglerverstärkung K_r wird, desto kleiner wird die Abweichung. Eine vollständige Kompensation ist nur theoretisch möglich für $K_r \to \infty$. Praktisch ist ein P-Regler nicht geeignet für diese Aufgabe. Deswegen wird der P-Regler später durch einen integralen Anteil erweitert werden.

Der Regelkreis wurde mit den selben fünf Reglerverstärkungen wie in Aufgabe 1 modelliert. Der Code ist in 4.1.1 dargestellt. Abbildung 4 zeigt das Blockschaltbild des Regelkreises. Abbildung 5 zeigt die zeitlichen Verläufe der Signale für die verschiedenen Reglerverstärkungen. Abbildung 6 vergleicht die stationären Endwerte. Die

stationären Endwerte und die Regelabweichungen werden in Tabelle 2 dargestellt..

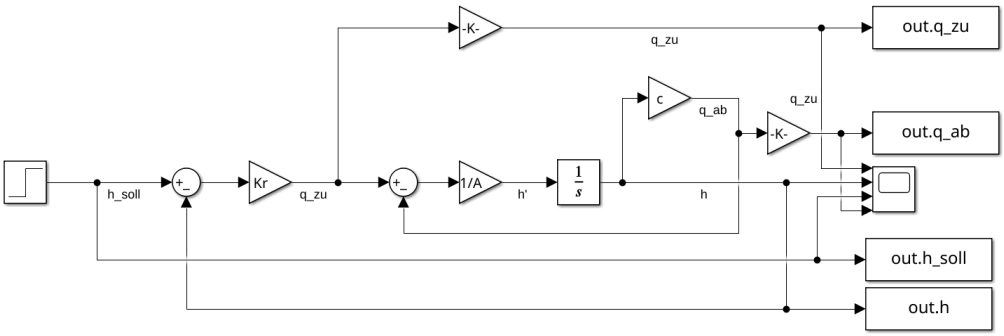

Abbildung 4: Blockschaltbild (Screenshot aus *Simulink*)

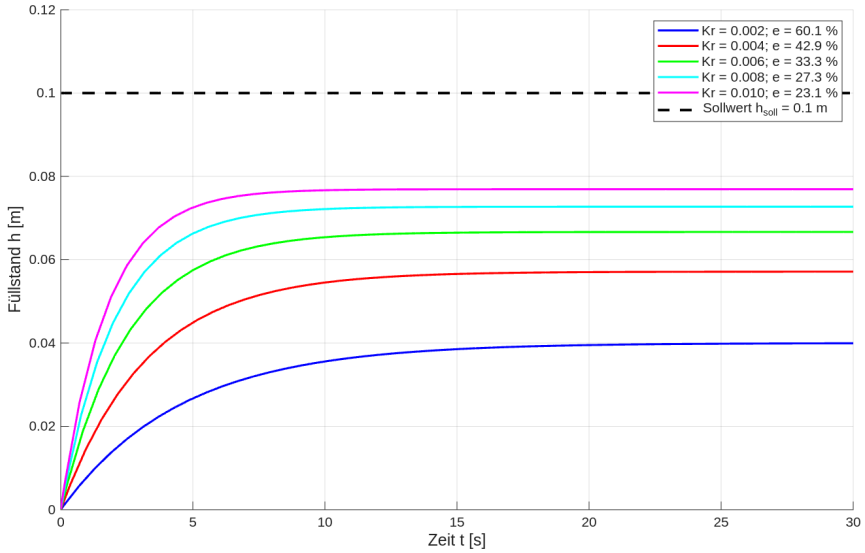

Abbildung 6: Vergleich der stationären Endwerte

Tabelle 2: Reglerverstärkungen und Regelabweichungen

Reglerverstärkung K_r	Stationärer Endwert $h(t \to \infty)$ [m]	Absolute Regelabweichung e_{abs} [m]	Relative Regelabweichung e_{rel} [%]
0.010	0.0766	0.0234	23.39
0.008	0.0720	0.0280	27.96
0.006	0.0651	0.0349	34.87
0.004	0.0541	0.0459	45.87
0.002	0.0352	0.0648	64.82

Quellcode 4.1.1: *Matlab*-Code zur Simulation und Berechung der stationären Regelabweichung

```
% Regelgröße: h, Führungsgröße: h_soll, Stellgröße: q_zu, Störgröße: q_ab
Kr_vector = [0.002, 0.004, 0.006, 0.008, 0.01]; % Reglerverstärkungen
n_sim = length(Kr_vector);

% Initialisierung für Ergebnisse
stationary_error_percent = zeros(1, n_sim);
stationary_error_abs = zeros(1, n_sim);
steady_state_h = zeros(1, n_sim);

open_system('a_modell_mit_abfluss');

% Simulationszeit
run_time = 30;
set_param('a_modell_mit_abfluss', 'StopTime', 'run_time');
set_param('a_modell_mit_abfluss', 'StartTime', '0');

% Step-Block
set_param('a_modell_mit_abfluss/Step', 'Time', '0');
set_param('a_modell_mit_abfluss/Step', 'Before', '0');
set_param('a_modell_mit_abfluss/Step', 'After', num2str(h_soll_wert));

for i = 1:n_sim
    Kr_actual = Kr_vector(i);

    assignin('base', 'Kr', Kr_actual);
    assignin('base', 'c', c);
    assignin('base', 'A', A);

    simOut = sim("a_modell_mit_abfluss.slx");

    if isstruct(simOut)
        t = simOut.tout;
        h = simOut.h;
        h_soll = simOut.h_soll;
        q_zu = simOut.q_zu;
        q_ab = simOut.q_ab;
    else
        t = simOut.get('tout');
        h = simOut.get('h');
        h_soll = simOut.get('h_soll');
        q_zu = simOut.get('q_zu');
        q_ab = simOut.get('q_ab');
    end

    sim_data{i}.t = t;
    sim_data{i}.h = h;
    sim_data{i}.h_soll = h_soll;
    sim_data{i}.q_zu = q_zu;
    sim_data{i}.q_ab = q_ab;
    sim_data{i}.Kr = Kr_actual;

    % Stationäre Regelabweichung
    n_samples = length(h);
    steady_start = round(0.9 * n_samples);
    steady_state_h(i) = mean(h(steady_start:end));
    stationary_error_abs(i) = h_soll_wert - steady_state_h(i);
    stationary_error_percent(i) = (stationary_error_abs(i) / h_soll_wert) * 100;

end
```

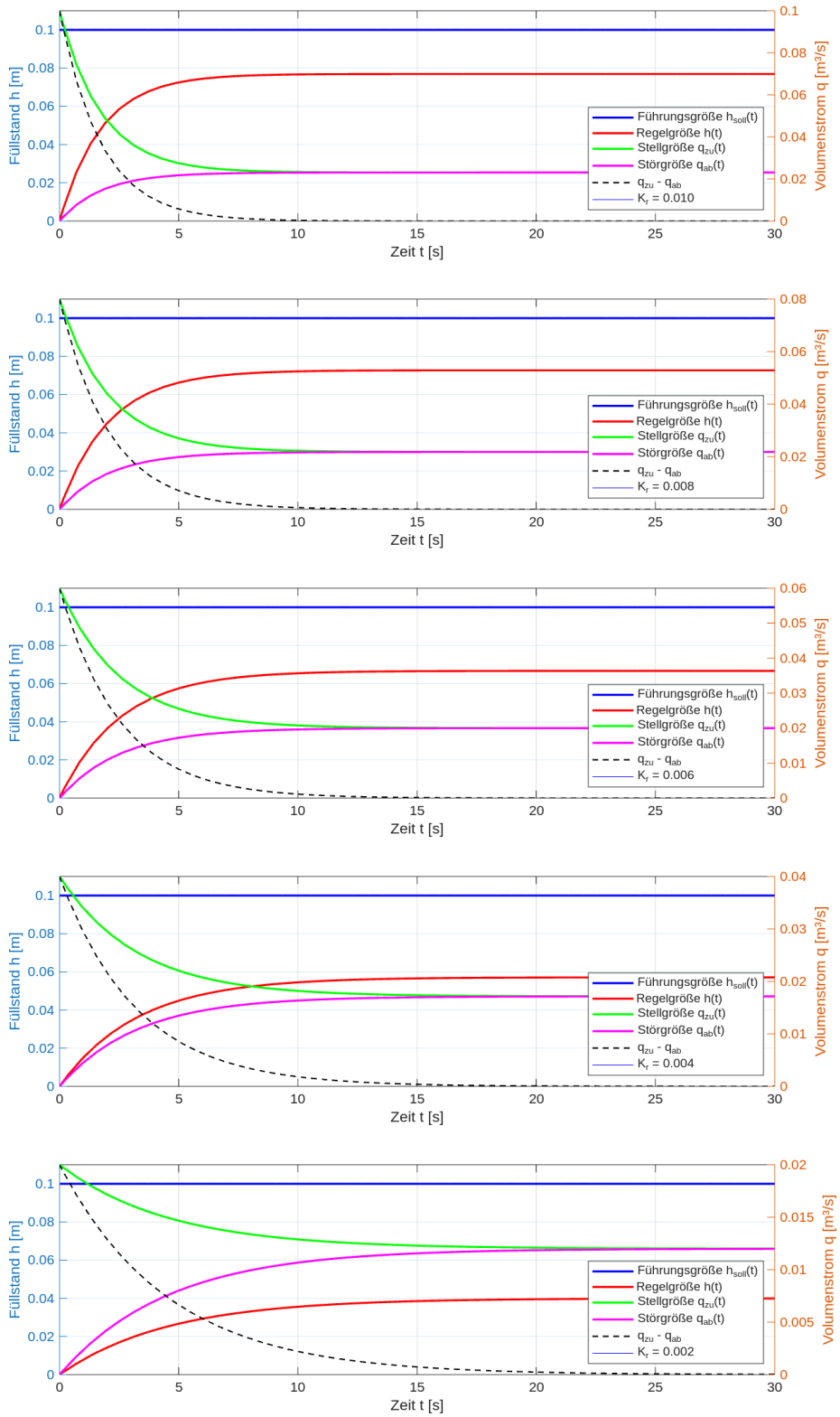

Abbildung 5: *Matlab*-Plots für verschiedene Reglerverstärkungen

4.2 Linearisiertes Ausflussgesetz mit PI-Regler

Ein besseres Modell erhält man, indem das Proportional-Glied K_r des Regelkreises um einen integrativen Anteil erweitert wird. Aus dem P-Regler wird so ein PI-Regler. Das Verhalten eines PI-Reglers wird durch Gleichung (7) beschrieben.

$$u(t) = K_r \cdot \left(e(t) + \frac{1}{T_n} \int e(t)\, dt \right) \tag{7}$$

Zusätzlich zu der Reglerverstärkung K_r muss nun auch die Nachstellzeit T_n berücksichtig werden. In *Simulink* wird ein PID-Regler ausgewählt und als PI-Regler definiert. Die Regelparameter werden nach Gleichung (8) festgelegt.

$$P = K_r \quad \text{und} \quad I = \frac{1}{T_n} \tag{8}$$

Für diese Simulation wird die Nachstellzeit variiert. Für die Reglerverstärkung wird $K_r = 0.006$ ausgewählt. Für große Nachstellzeiten gibt es keine Überschwinger. Je kleiner die Nachstellzeit ist, desto mehr wird der integrale Anteil bezogen auf den proportionalen Anteil gewichtet und die Überschwinger nehmen zu. Gesucht ist die Nachstellzeit, für die es keine Überschwinger mehr gibt. Das Intervall wird in *Matlab* iterativ gesucht. Als Erstes werden Nachstellzeiten zwischen 100 s und $0{,}01$ s untersucht. Das Intervall wird immer kleiner gewählt, um einen möglichst exakten Wert für T_n zu ermitteln. Für $T_n \geq 6{,}2$ s gibt es keine Überschwinger mehr.

Der *Matlab*-Code ist in 4.2.1 und 4.2.2. Abbildung 7 zeigt das Modell. In der Abbildung 8 sind die zeitlichen Verläufe der Signale für verschiedene Nachstellzeiten in dem exakten Intervall dargestellt. Dieses Intervall wird mit den jeweiligen Überschwingern in Abbildung 9 gezeigt. Tabelle 3 zeigt die Überschwinger für verschiedene Nachstellzeiten und die iterativen Intervalle für die Ermittelung von T_n.

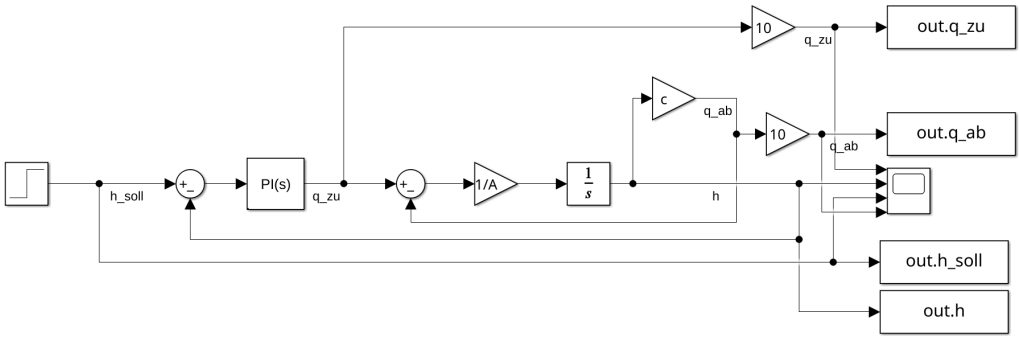

Abbildung 7: Blockschaltbild (Screenshot aus *Simulink*)

Quellcode 4.2.1: *Matlab*-Code zur Simulation und Berechnung der Überschwinger

```
assignin('base', 'A', A);
assignin('base', 'c', c);
assignin('base', 'Kr', Kr_fixed);

%% Nachstellzeiten
%Tn_vector = [100, 50, 20, 10, 5, 2, 1, 0.5, 0.2, 0.1, 0.05, 0.02, 0.01];
Tn_vector = [6.2, 6.15, 6.1, 6.05, 6];
n_sim = length(Tn_vector);

% Initialisierung
overshoot_percent = zeros(1, n_sim);
has_overshoot = zeros(1, n_sim);
steady_state_h = zeros(1, n_sim);
max_h_values = zeros(1, n_sim);

%% Simulationsparameter
run_time = 30;
open_system('b_modell_mit_abfluss');
set_param('b_modell_mit_abfluss', 'StopTime', num2str(run_time));
set_param('b_modell_mit_abfluss', 'StartTime', '0');
set_param('b_modell_mit_abfluss/Integrator', 'InitialCondition', '0');

% Step-Block
set_param('b_modell_mit_abfluss/Step', 'Time', '0');
set_param('b_modell_mit_abfluss/Step', 'Before', '0');
set_param('b_modell_mit_abfluss/Step', 'After', num2str(h_soll_wert));

for i = 1:n_sim
    Tn = Tn_vector(i);
    Ki = 1/Tn;

    set_param('b_modell_mit_abfluss/PID Controller', 'P', num2str(Kr_fixed));
    set_param('b_modell_mit_abfluss/PID Controller', 'I', num2str(Ki));
    set_param('b_modell_mit_abfluss/PID Controller', 'D', '0');

    simOut = sim("b_modell_mit_abfluss.slx");

    if isstruct(simOut)
        t = simOut.tout;
        h = simOut.h;
        q_zu = simOut.q_zu;
        q_ab = simOut.q_ab;
        h_soll = simOut.h_soll;
    else
        t = simOut.get('tout');
        h = simOut.get('h');
        q_zu = simOut.get('q_zu');
        q_ab = simOut.get('q_ab');
        h_soll = simOut.get('h_soll');
    end

    if isa(h, 'timeseries')
        h = h.Data;
        t = t.Data;
        q_zu = q_zu.Data;
        q_ab = q_ab.Data;
    end

```

Quellcode 4.2.2: *Matlab*-Code zur Simulation und Berechnung der Überschwinger

```

        % Stationärer Endwert
        n_end = round(0.9 * length(h));
        h_end = mean(h(n_end:end));
        steady_state_h(i) = h_end;

        % Maximalen Wert finden
        idx_start = find(t > 2, 1, 'first');
        if isempty(idx_start)
            idx_start = 1;
        end
        h_max = max(h(idx_start:end));
        max_h_values(i) = h_max;

        % Überschwinger
        if h_max > h_end * 1.01
            overshoot_percent(i) = (h_max - h_end) / h_end * 100;
            has_overshoot(i) = 1;
            fprintf('  -> ÜBERSCHWINGER: %.2f%% (h_max = %.4f m, h_end = %.4f m)\n'
            , ...
                    overshoot_percent(i), h_max, h_end);
        else
            overshoot_percent(i) = 0;
            has_overshoot(i) = 0;
            fprintf('  -> Kein Überschwinger (h_max = %.4f m, h_end = %.4f m)\n',
            h_max, h_end);
        end

...

        % Intervallbestimmung
        Tn_overshoot = Tn_vector(has_overshoot == 1);
        Tn_no_overshoot = Tn_vector(has_overshoot == 0);

        if ~isempty(Tn_no_overshoot) && ~isempty(Tn_overshoot)
            Tn_max_no_overshoot = max(Tn_no_overshoot);
            Tn_min_overshoot = min(Tn_overshoot);

            fprintf('  Keine Überschwinger für TN   %.4f s\n',
            Tn_max_no_overshoot);
            fprintf('  Überschwinger vorhanden für TN   %.4f s\n', Tn_min_overshoot);
            fprintf('  Grenzbereich: %.4f s < TN_kritisch < %.4f s\n',
            Tn_max_no_overshoot, Tn_min_overshoot);

        elseif isempty(Tn_no_overshoot)
            fprintf('  Bei allen getesteten TN-Werten treten Überschwinger auf!\n');
        elseif isempty(Tn_overshoot)
            fprintf('  Bei keinem getesteten TN-Wert treten Überschwinger auf!\n');
        end
```

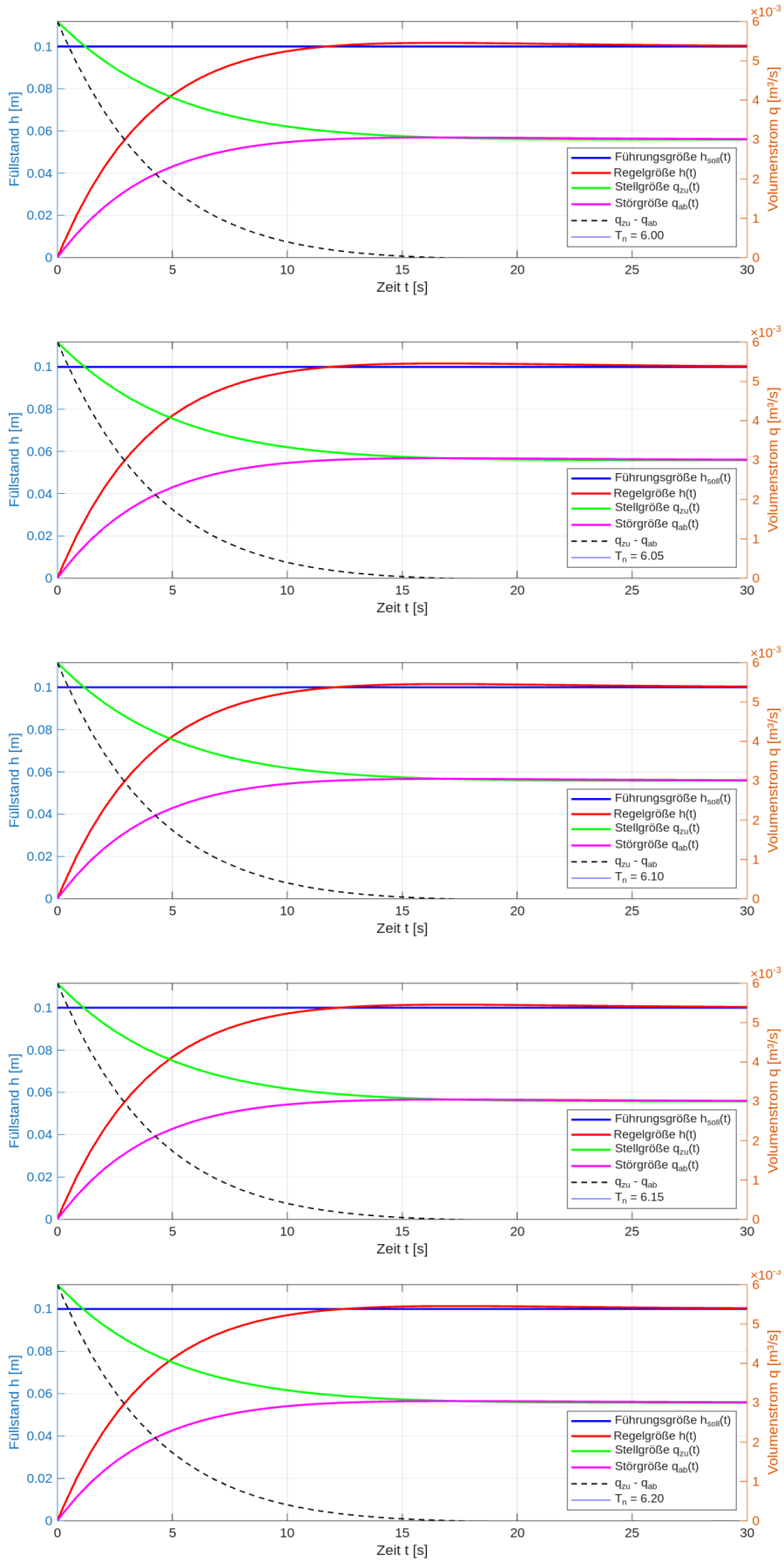

Abbildung 8: *Matlab*-Plots für verschiedene Rückstellzeiten

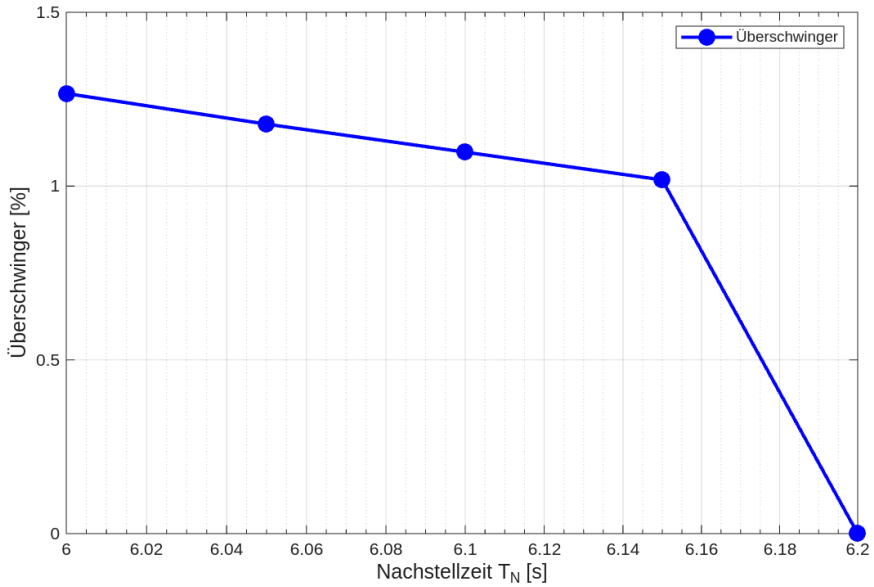

Abbildung 9: Vergleich der Überschwinger

Tabelle 3: Reglerverstärkungen und Regelabweichungen

Rückstellzeit T_n [s]	Überschwinger [%]
100	0.00
50	0.00
20	0.00
10	0.00
9	0.00
8	0.00
7	0.00
6.8	0.00
6.6	0.00
6.4	0.00
6.2	0.00
6	1.20
5	3.51
2	18.23
1	31.67
0.5	43.81
0.2	58.45
0.1	64.30
0.05	71.86
0.02	54.80
0.01	64.00

4.3 Ausflussgesetz nach Torricelli

Eine physikalisch exakte Beschreibung des Ausflussverhaltens von Wasser liefert der Satz von Torricelli. Demnach hängt die Ausflussgeschwindigkeit und damit der Abfluss nur vom Füllstand ab. Der mathematische Zusammenhang ist in Gleichung (9) dargestellt. Es wird gefordert, dass sich bei einem Füllstand von $h = 5\,\text{cm}$ ein Abfluss von $q_{ab} = 0{,}3\,\frac{\text{l}}{\text{s}}$ ergibt.

$$q_{ab}(t) = k \cdot \sqrt{h(t)} \quad \Longrightarrow \quad k = \frac{q_{ab}(t)}{\sqrt{h(t)}} = 0.0013416 \tag{9}$$

In Abbildung 7 wird der Ausgang der Regelstrecke über den proportionalen Faktor c rückgekoppelt. Dieser Faktor muss entsprechend des Satzes von Torricelli erweitert werden. Zuerst wird der Betrag des Signals gebildet, um negative Eingänge für den nachfolgenden Wurzelblock zu vermeiden. Danach wird das Signal um den Faktor k verstärkt. Der Code für die Simulation ist in 4.3.1 zu finden. Das erweiterte Modell ist in Abbildung 10 abgebildet. Die Simulation arbeitet mit den selben Werte für die Reglerverstärkung und die Nachstellzeit: $K_r = 0{,}006$ und $T_n = 6{,}2\,\text{s}$. Der zeitliche Verlauf der Signale ist in Abbildung 11 abgebildet.

Der PI-Regler zeigt auch mit dem nichtlinearen Torricelli-Abflussgesetz eine sehr gute stationäre Genauigkeit. Der Füllstand erreicht einen stationären Endwert von $h(t \to \infty) = 10{,}05\,\text{cm}$. Die Regelabweichung beträgt nur 0.48%, was praktisch einer exakten Regelung entspricht. Im Vergleich zum linearen Abflussmodell zeigt das Torricelli-Modell ein verändertes Einschwingverhalten. Die nichtlineare Kennlinie führt zu einer geringeren Dämpfung, was sich in einem größeren Überschwinger bemerkbar macht. Mit den übernommenen Reglerparametern treten im Torricelli-Fall deutliche Überschwinger auf. Dies liegt daran, dass der nichtlineare Abfluss bei niedrigem Füllstand geringer ist als im linearen Modell, wodurch der Regler zunächst zu stark regelt.

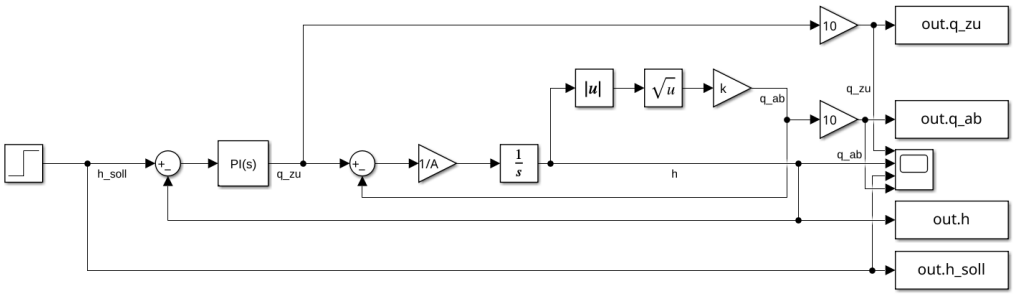

Abbildung 10: Blockschaltbild (Screenshot aus *Simulink*)

Quellcode 4.3.1: *Matlab*-Code zur Simulation mit dem Torricelli-Ausflussgesetz

```
assignin('base', 'k', k);
assignin('base', 'A', A);
assignin('base', 'Kr', Kr_fixed);
assignin('base', 'Ki', Ki);
assignin('base', 'h_soll_wert', h_soll_wert);

%% Simulationsparameter
run_time = 30;
open_system('c_modell_mit_abfluss');
set_param('c_modell_mit_abfluss', 'StopTime', 'run_time');
set_param('c_modell_mit_abfluss', 'StartTime', '0');

% Step-Block
set_param('c_modell_mit_abfluss/Step', 'Time', '0');
set_param('c_modell_mit_abfluss/Step', 'Before', '0');
set_param('c_modell_mit_abfluss/Step', 'After', num2str(h_soll_wert));

simOut = sim("c_modell_mit_abfluss.slx");

%% Daten auslesen
if isstruct(simOut)
    t = simOut.tout;
    h = simOut.h;
    q_zu = simOut.q_zu;
    q_ab = simOut.q_ab;
    h_soll = simOut.h_soll;
else
    t = simOut.get('tout');
    h = simOut.get('h');
    q_zu = simOut.get('q_zu');
    q_ab = simOut.get('q_ab');
    h_soll = simOut.get('h_soll');
end

% Bei timeseries-Objekten: Daten extrahieren
if isa(h, 'timeseries')
    h = h.Data;
    t = t.Data;
    q_zu = q_zu.Data;
    q_ab = q_ab.Data;
    h_soll = h_soll.Data;
end

%% Stationärer Endwert
n_end = round(0.9 * length(h));
h_end = mean(h(n_end:end));
q_ab_end = mean(q_ab(n_end:end));
q_zu_end = mean(q_zu(n_end:end));

fprintf('Stationäre Werte (letzte 10%%):\n');
fprintf('  h(∞) = %.4f m (%.1f cm)\n', h_end, h_end*100);
fprintf('  q_ab(∞) = %.6f m³/s (%.3f L/s)\n', q_ab_end, q_ab_end*1000);
fprintf('  q_zu(∞) = %.6f m³/s (%.3f L/s)\n', q_zu_end, q_zu_end*1000);
```

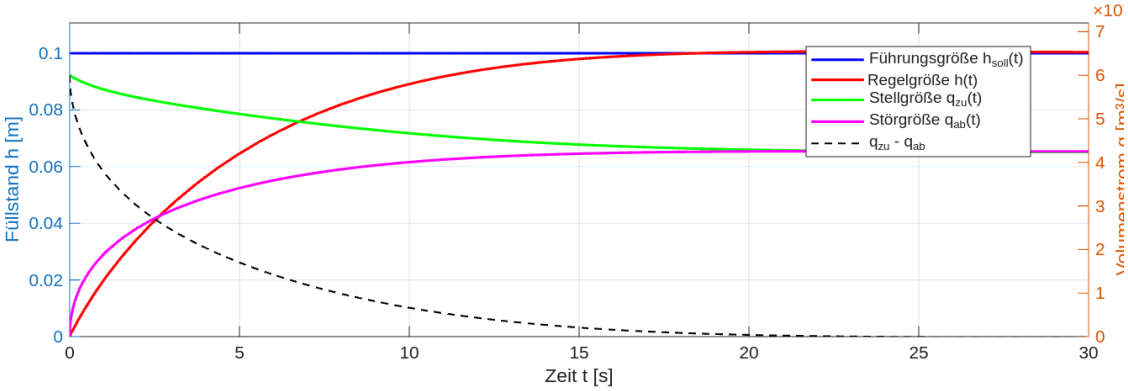

Abbildung 11: *Matlab*-Plot für $K_r = 0.006$ und $T_n = 6.2s$